

[← Back to Blog](#)[A2A Protocol](#) [AI](#) [Agent Communication](#) [JSON-RPC](#)

What is A2A? Understanding the Agent-to-Agent Protocol

Phoenix • October 20, 2025 • 8 min read

What is A2A? Understanding the Agent-to-Agent Protocol

In the rapidly evolving world of artificial intelligence, we're witnessing an explosion of AI agents - specialized systems designed to perform specific tasks, from answering questions to managing workflows. But as these agents multiply across different platforms and frameworks, a critical challenge emerges: **How do these agents communicate with each other?**

Enter the **Agent-to-Agent (A2A) protocol** - a standardized communication layer that's revolutionizing how AI agents interact across different platforms, frameworks, and organizations.

The Problem: Fragmented AI Ecosystems

Imagine you've built an amazing AI agent using TypeScript and Mastra. Your colleague built another one using Python and LangChain. Your organization wants to use both in a unified workflow platform like Telex. Without a standard protocol, you'd face:

- **Proprietary APIs:** Each agent speaks its own language
- **Complex Integrations:** Custom code for every agent-to-platform connection
- **Limited Interoperability:** Agents can't easily work together
- **Vendor Lock-in:** Switching platforms means rewriting everything

This fragmentation severely limits the potential of multi-agent systems.

The Solution: A2A Protocol

The Agent-to-Agent (A2A) protocol provides a standardized way for AI agents to communicate, regardless of their underlying implementation. Think of it as HTTP for AI agents - just as HTTP enables web browsers to talk to any web server, A2A enables any agent to talk to any other agent or platform.

Core Principles

The A2A protocol is built on several key principles:

1. **Standardization:** A consistent message format across all implementations
2. **Language Agnostic:** Works with Python, TypeScript, Go, or any language
3. **Platform Independent:** Not tied to any specific framework or cloud provider
4. **Stateful:** Maintains conversation context and history
5. **Production Ready:** Built on proven standards like JSON-RPC 2.0

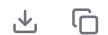
How A2A Works

At its core, A2A is built on the **JSON-RPC 2.0** specification, a lightweight remote procedure call protocol. Here's what a typical A2A interaction looks like:

1. The Request

When you want to send a message to an agent, you construct an A2A request:

json



```
{
  "jsonrpc": "2.0",
  "id": "request-123",
  "method": "message/send",
  "params": {
    "message": {
      "kind": "message",
      "role": "user",
      "parts": [
        {
          "kind": "text",
          "text": "What's the weather in Lagos?"
        }
      ]
    },
    "messageId": "msg-001",
    "taskId": "task-001"
  },
  "configuration": {
    "blocking": true
  }
}
```

```
}
```

2. The Response

The agent processes your request and returns a standardized response:

json



```
{
  "jsonrpc": "2.0",
  "id": "request-123",
  "result": {
    "id": "task-001",
    "contextId": "ctx-uuid",
    "status": {
      "state": "completed",
      "timestamp": "2025-10-20T10:30:00.000Z",
      "message": {
        "messageId": "msg-002",
        "role": "agent",
        "parts": [
          {
            "kind": "text",
            "text": "The current weather in Lagos is 29°C with thunderstorms."
          }
        ],
        "kind": "message"
      }
    },
    "artifacts": [
      {
        "artifactId": "artifact-uuid",
        "name": "weatherData",
        "parts": [
          {
            "kind": "data",
            "data": {
              "temperature": 29.1,
              "humidity": 85,
              "conditions": "Thunderstorm"
            }
          }
        ]
      }
    ],
    "history": [...],
    "kind": "task"
  }
}
```

Key Components of A2A

Messages

Messages are the fundamental unit of communication in A2A. Each message contains:

- **Role:** Who sent the message (user, agent, system)
- **Parts:** The actual content (text, data, files)
- **Metadata:** IDs, timestamps, and context information

Message Parts

A2A supports multiple content types within a single message:

typescript



```
{
  "kind": "text",
  "text": "Here's the analysis..."
}

{
  "kind": "data",
  "data": { "temperature": 29, "humidity": 85 }
}

{
  "kind": "file",
  "file_url": "https://storage.example.com/image.png"
}
```

This flexibility enables rich, multimedia interactions between agents.

Task Management

A2A introduces the concept of **tasks** - discrete units of work with their own lifecycle:

- **Task ID:** Unique identifier for tracking
- **Context ID:** Groups related tasks into conversations
- **Status:** Current state (working, completed, input-required, failed)
- **History:** Complete conversation trail

Artifacts

Artifacts are structured outputs from agent executions. They can include:

- Analysis results
- Generated code
- Images or visualizations
- Structured data
- File references

Artifacts make it easy to chain agents together - one agent's output becomes another's input.

Why A2A Matters

1. Interoperability

With A2A, a Python agent can seamlessly communicate with a TypeScript agent, which can interact with a Go agent, all within the same workflow. No custom integration code required.

2. Modularity

Build specialized agents for specific tasks:

- Weather agent handles meteorological data
- Translation agent handles languages
- Analysis agent processes datasets
- Code agent writes and reviews code

Then compose them into powerful workflows without worrying about compatibility.

3. Vendor Independence

Your agents aren't locked into a specific platform. An agent built for Mastra can run on Telex, or any other A2A-compliant platform, with zero code changes.

4. Ecosystem Growth

As more developers adopt A2A, we'll see an explosion of reusable agents:

- Public agent marketplaces
- Pre-built domain specialists
- Community-contributed tools

- Enterprise agent libraries

Real-World Use Cases

Customer Support Workflow



User Query → Routing Agent → Knowledge Base Agent →
↓ (if complex)
Human Handoff Agent → Support Team

Content Creation Pipeline



Brief → Research Agent → Writing Agent → Review Agent →
Publishing Agent → SEO Analysis Agent

Data Processing System



Raw Data → Validation Agent → Transformation Agent →
Analysis Agent → Visualization Agent → Report Agent

Development Assistant



User Request → Code Generation Agent → Testing Agent →
Review Agent → Deployment Agent

A2A vs. Traditional APIs



Aspect	Traditional API	A2A Protocol
Message Format	Proprietary	Standardized
Conversation Context	Manual tracking	Built-in
Multi-turn Interactions	Complex state management	Native support
Interoperability	Limited	High
Task Lifecycle	Application-specific	Standardized states
Error Handling	Varies by API	JSON-RPC 2.0 standard
Artifacts	Not standardized	Structured format

Building A2A Agents

The beauty of A2A is that you can implement it in any language or framework:

TypeScript with Mastra

typescript



```
import { Agent } from '@mastra/core/agent';

const agent = new Agent({
  name: 'My Agent',
  instructions: 'You are a helpful assistant...',
  model: 'gpt-4',
  tools: { myTool }
});

// A2A endpoint automatically exposed
```

Python with FastAPI

python



```
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()

@app.post("/a2a/agent")
async def a2a_endpoint(request: JSONRPCRequest):
    # Process A2A request
    # Return A2A response
    pass
```

Any Language

As long as you can:

1. Accept HTTP POST requests
2. Parse JSON-RPC 2.0 format
3. Return A2A-compliant responses

You can build A2A agents in Go, Rust, Java, Ruby, or any language.

The A2A Specification

The A2A protocol is an open specification, meaning:

- **Open Source:** The spec is publicly available
- **Community Driven:** Contributions and improvements are welcome
- **Extensible:** Add custom fields while maintaining compatibility
- **Well Documented:** Clear examples and reference implementations

Deployment Models

A2A agents support multiple deployment patterns:

Blocking (Synchronous)

Agent processes request immediately and returns result:

json



```
{
  "configuration": {
    "blocking": true
  }
}
```



```
}
```

Webhook (Asynchronous)

Agent accepts request, processes in background, and sends result to webhook:

json



```
{
  "configuration": {
    "blocking": false,
    "pushNotificationConfig": {
      "url": "https://platform.example.com/webhook",
      "token": "auth-token"
    }
  }
}
```

This flexibility supports both real-time and long-running operations.

Getting Started with A2A

Ready to build your first A2A agent? Here's the path:

1. **Choose Your Stack:** Pick a language and framework
2. **Implement the Protocol:** Follow the A2A specification
3. **Test Your Agent:** Use curl or Postman to send A2A requests
4. **Deploy:** Host your agent on any platform
5. **Integrate:** Connect to A2A-compliant workflow platforms

The Future of Multi-Agent Systems

The A2A protocol represents a fundamental shift in how we think about AI agents. Instead of monolithic, do-everything agents, we're moving toward:

- **Specialized Agents:** Each agent excels at one thing
- **Composable Workflows:** Combine agents like building blocks
- **Agent Marketplaces:** Discover and use community agents

- **Cross-Platform Collaboration:** Agents work together regardless of origin

As the AI landscape continues to evolve, standardized communication protocols like A2A will be crucial for building scalable, maintainable, and interoperable AI systems.

Conclusion

The Agent-to-Agent protocol is more than just a technical specification - it's the foundation for the next generation of AI systems. By providing a standardized way for agents to communicate, A2A enables:

- True interoperability across platforms and languages
- Modular, composable agent architectures
- Rapid development and deployment of specialized agents
- Rich ecosystem of reusable AI components

Whether you're building a single agent or orchestrating complex multi-agent workflows, understanding A2A is essential for modern AI development.

In our next posts, we'll dive deeper into:

- Building A2A agents with Python and FastAPI
- Integrating Mastra agents with Telex using A2A
- Advanced A2A patterns and best practices

Resources:

- [A2A Protocol Specification](#)
- [JSON-RPC 2.0 Specification](#)
- [Mastra Documentation](#)
- [Example A2A Implementations](#)

Welcome to the future of agent communication.

← Back to all posts

Share on X

PHOENIX